

DSVW Business Rules & Security Requirements

Testing document for the Damn Small Vulnerable Web (DSVW) repository

Purpose

This document provides business and security requirements that can be uploaded to AI Code Guardian as a Business Intent document while scanning DSVW. The rules are intentionally written as organizational requirements so the Business Intent Engine can compare repository behavior against expected secure-development practices.

DSVW is a deliberately vulnerable Python web application created for education and security testing. Its upstream project states that it supports many common web-application vulnerabilities and associated attack cases. Use this document only for authorized/local testing of the DSVW project.

Business Context

Field	Requirement
Application type	Educational web security testing application
Primary objective	Demonstrate and test secure coding controls against common web vulnerabilities
Security posture	Production security controls are expected to be applied to normal application code
Input trust	All data received from users, HTTP requests, files, URLs, XML, and external services must be treated as untrusted
Data protection	Sensitive or security-relevant operations must not rely on attacker-controlled input
Availability	Security controls must not introduce avoidable denial-of-service risks
Auditability	Security-sensitive actions should be understandable and traceable

Business & Security Rules

These rules are the primary Business Intent test set. A rule violation should be reported only when repository evidence supports it. Do not infer a violation solely from the rule's presence.

BR-001	Untrusted Input Validation Critical
Requirement	All externally supplied input must be treated as untrusted and validated before it reaches security-sensitive operations. Validation should enforce expected type, format, length, and allowed values where applicable.
Evidence terms	HTTP parameters; request bodies; user-controlled values; validation; allowlist
BR-002	Safe Deserialization Critical
Requirement	The application must not deserialize attacker-controlled data using mechanisms that can instantiate arbitrary objects or execute unintended behavior. Only explicitly permitted data formats and structures should be accepted.
Evidence terms	deserialization; serialized objects; trusted format; object reconstruction; untrusted data
BR-003	Parameterized Database Queries Critical
Requirement	Database queries must separate code from user-provided data. Application input must never be directly concatenated into SQL statements.
Evidence terms	SQL query; parameterized query; prepared statement; database input

BR-004	Safe Command Execution Critical
Requirement	User-controlled input must never be directly interpreted as an operating-system command. If an operating-system operation is necessary, use restricted commands and safe argument handling.
Evidence terms	command execution; subprocess; shell; OS command; safe arguments
BR-005	Output Encoding High
Requirement	Data originating from users or other untrusted sources must be safely encoded for its output context before being rendered in a web response.
Evidence terms	HTML output; encoding; escaping; untrusted data; browser response
BR-006	Safe XML Processing High
Requirement	XML supplied from untrusted sources must be processed using secure parser settings that prevent unsafe external entity resolution and related resource access.
Evidence terms	XML; parser; external entity; entity resolution; secure parsing
BR-007	Safe XPath Handling High
Requirement	Values supplied by users must not be inserted directly into XPath expressions. Input must be validated or safely handled before use in XML queries.
Evidence terms	XPath; XML query; user input; parameterization; validation
BR-008	Restricted External Requests High
Requirement	URLs or hosts supplied by untrusted users must not be used for unrestricted outbound server-side requests. External destinations must be validated against an approved policy.
Evidence terms	URL; outbound request; external service; host validation; allowlist
BR-009	Controlled File Access High
Requirement	User-controlled paths must not be used to access arbitrary files. File operations must remain within approved application directories and use safe path validation.
Evidence terms	file path; file access; path validation; approved directory
BR-010	Authentication and Authorization High
Requirement	Security-sensitive operations must require appropriate authentication and authorization. Access decisions must not rely solely on user-supplied identifiers or client-controlled state.
Evidence terms	authentication; authorization; access control; protected operation
BR-011	Security-Safe Error Handling Medium
Requirement	Application errors must not expose sensitive implementation details, internal paths, credentials, or security-sensitive data to untrusted users.
Evidence terms	error handling; exception; stack trace; internal path
BR-012	Resource and Request Limits Medium
Requirement	Externally controlled operations should apply reasonable limits on input size, processing cost, recursion, file size, or request frequency where excessive resource use could affect availability.
Evidence terms	resource limits; request size; processing limit; timeout

BR-013	Secure Dependency Management Medium
Requirement	Runtime dependencies should use maintained versions and explicit version constraints appropriate for the application's security requirements.
Evidence terms	dependency; requirements; version pinning; maintained package
BR-014	Security Logging and Traceability Medium
Requirement	Security-relevant failures and sensitive operations should produce sufficient diagnostic information for investigation without logging secrets or other sensitive data.
Evidence terms	security event; logging; audit trail; diagnostic information
BR-015	Secure-by-Default Configuration Medium
Requirement	Security-sensitive functionality must use safe defaults. Optional security controls should not be disabled by default when doing so exposes the application to avoidable risk.
Evidence terms	configuration; secure defaults; security control; default behavior
BR-016	Cross-Site Request Protection High
Requirement	State-changing web operations should verify that requests originate from an expected application context and should use appropriate anti-forgery protections where applicable.
Evidence terms	state-changing request; CSRF; anti-forgery; request origin

Expected Business Intent Interpretation

When AI Code Guardian scans DSVW with this document, the Business Intent Engine should compare detected code behavior against these expectations. Because DSVW is intentionally vulnerable, the purpose of the test is to produce meaningful misalignment findings rather than to make the repository appear compliant.

Area	Expected interpretation
Input handling	Potential violations are expected where untrusted values reach sensitive operations without adequate validation.
Deserialization	Unsafe deserialization behavior should be treated as a major business-security mismatch.
Database access	Direct construction of database queries from untrusted values should be flagged.
Command execution	User-controlled values reaching OS command execution should be considered high risk.
Browser output	Untrusted values rendered without context-appropriate encoding should be treated as a security mismatch.
XML / XPath	Unsafe XML entity handling or direct construction of XPath expressions from untrusted data should be flagged when ev
Outbound requests	Unrestricted user-controlled destinations should be treated as a server-side request security concern.
File access	Unrestricted user-controlled file paths should be treated as a controlled-file-access violation.
Authentication / authorization	Security-sensitive functionality lacking appropriate access controls should be flagged where evidence exists.
Dependencies	Dependency findings should be separated from business-intent findings unless they directly violate a stated requirement

Testing Notes for AI Code Guardian

- **Use repository-relative evidence.** Findings should point to the relevant file and line when available.
- **Do not fabricate business impact.** If a rule is violated but business context is unknown, state that the business impact could not be determined.
- **Separate detection from inference.** A deterministic security finding is evidence; an agentic explanation should clearly indicate when reachability or impact is inferred.
- **Keep uncertainty visible.** Use labels such as Potential, Heuristic, Not Confirmed, or Insufficient Evidence when appropriate.
- **Remediation must be grounded.** A suggested fix should be tied to the detected code and should not invent dependency versions or unrelated code changes.
- **Validation is separate from remediation.** A proposed fix should not be described as fixed unless the validation layer confirms it.

Source & Scope

The DSVW upstream project describes it as a deliberately vulnerable web application for educational purposes and security testing. Its README states that it supports many common web-application vulnerabilities and includes regression tests for its listed attack cases. This PDF is a purpose-built Business Intent test pack; it is not an official DSVW requirements document.

Recommended use: upload this PDF as the Business Intent / Business Rules document for a controlled AI Code Guardian scan of the DSVW repository. The application should remain isolated/local because it is intentionally vulnerable.